

Optimisation du transport universitaire

Test technique MTDI Bénin

Hypothèses du jeu de données simulé et réponses à « Pour aller plus loin »

- Document complémentaire au README et au dépôt de code
- 16 arrêts · 6 lignes · 20 bus · environ 8 520 déplacements simulés par jour
- Données simulées, graine reproductible (SEED=2026)

1. Hypothèses du jeu de données simulé

Aucun jeu de données n'étant fourni par l'énoncé, les données sont entièrement générées par un moteur de simulation. Cette section documente **tous les types d'hypothèses** retenues : réseau, distances et vitesses, profil horaire de la demande, et génération individuelle des étudiants. Toutes ces valeurs sont réunies dans `server/src/config.js` et `server/src/data/`, et affichées dans le tableau de bord de l'application.

1.1 Hypothèses de réseau (arrêts et lignes)

16 arrêts répartis entre Abomey-Calavi et Cotonou, avec pour chacun un type (résidence, carrefour, campus), une **popularité** (poids relatif dans la génération de la demande) et une distance de marche moyenne des étudiants qui l'utilisent. 6 lignes ont été définies, chacune reliant une zone résidentielle à un pôle universitaire (dernier arrêt de la ligne).

Ligne	Itinéraire	Bus × places	Longueur	Étudiants / jour
L1	Calavi → ENEAM → UAC	3 × 60	7,1 km	2 500
L2	Akpakpa → Godomey → UAC	6 × 60	20,8 km	2 000
L3	Fidjrossè → Godomey → UAC	4 × 60	15,7 km	1 500
L4	Dantokpa → Cadjèhoun → Campus Cotonou	3 × 60	9,6 km	1 300
L5	Agla → Campus Cotonou	1 × 30	4,7 km	220
L6	Calavi → Cadjèhoun → Campus Cotonou	3 × 60	20,1 km	1 000

Total : 20 bus, 1 170 places, environ 8 520 déplacements d'étudiants par jour, service de 05h00 à 21h00. La ligne L1 reprend l'exemple de l'énoncé (2 500 étudiants/jour). La ligne L5 est volontairement sous-utilisée pour tester la détection des lignes à fusionner (cas d'usage 5).

1.2 Hypothèses de distance et de vitesse

— **Distance routière** — distance à vol d'oiseau (formule de Haversine) multipliée par un coefficient de détour de **1,3**, pour approcher une distance réelle sans routage détaillé.

— **Vitesse selon l'heure** — 32 km/h à 5h, **15–16 km/h aux heures de pointe** (7–8h et 17–18h), 20 km/h vers midi et à 16h, 22–30 km/h le reste de la journée : la circulation ralentit aux heures de pointe, comme observé en zone urbaine dense.

— **Temps d'arrêt** — 25 secondes de base + 1,5 seconde par passager qui monte ou descend (plafonné à 180 s), avec une pause de 2 minutes au terminus.

— **Marche à pied** — vitesse de 4,5 km/h avec un coefficient de détour de 1,25 (rues, trottoirs).

1.3 Hypothèses de profil horaire de la demande

La part de la demande journalière par heure a été estimée à partir d'un profil universitaire type (pointe du matin marquée, petite pointe de midi, pointe du soir plus étalée). Les parts sont normalisées pour faire 100 % sur la plage de service (05h–21h).

Heure	Part de la demande	Sens dominant vers le campus
05h – 06h	2 % – 7 %	88 % – 90 %
07h – 08h (pointe matin)	14 % – 13 %	85 % – 90 %
09h – 11h	6 % – 5 %	45 % – 60 %
12h (pointe midi)	7 %	50 %
13h – 16h	5 % – 6 %	30 % – 45 %

Heure	Part de la demande	Sens dominant vers le campus
17h – 18h (pointe soir)	8 % – 6 %	12 % – 15 %
19h – 20h	3 % – 2 %	15 % – 20 %

Cette asymétrie du sens des trajets (très majoritairement vers le campus le matin, l'inverse le soir) est l'hypothèse la plus structurante du modèle : c'est elle qui explique pourquoi les lignes sont saturées à plus de 85 % dans un sens en pointe, tout en étant remplies à moins de 20 % en moyenne sur la journée (les bus roulant à vide dans le sens contraire).

1.4 Hypothèses de génération individuelle des étudiants

- **Nombre d'arrivées** — à chaque pas de simulation de 10 secondes, le nombre de nouveaux étudiants par ligne suit une **loi de Poisson**, dont la moyenne dépend de la demande journalière de la ligne et de la part horaire ci-dessus.
- **Origine et destination** — le couple d'arrêts est tiré au hasard parmi tous les couples possibles de la ligne, pondéré par la popularité des deux arrêts (multipliée par 3 si la destination est un campus, pour refléter l'attractivité des pôles universitaires).
- **Distance de marche** — tirée selon une loi exponentielle centrée sur la distance moyenne de l'arrêt de départ (plafonnée à 2,5 km).
- **Reproductibilité** — l'ensemble du tirage aléatoire est piloté par un générateur à **graine fixe** (mulberry32, graine par défaut 2026) : les mêmes paramètres produisent toujours exactement les mêmes résultats, ce qui permet de comparer deux configurations sur des bases identiques et de vérifier automatiquement le comportement du moteur (tests unitaires).

1.5 Hypothèses propres à la recommandation (cas d'usage 4)

Seuils utilisés pour ramener chaque critère entre 0 et 1 dans la fonction de score (détaillée dans le README) : au-delà de **30 minutes** d'attente, la note d'attente vaut 0 ; au-delà de **8 places** libres, la note de places est plafonnée à 1 (avoir plus de places n'apporte plus rien) ; au-delà de **1,5 km** de marche, l'arrêt n'est plus proposé ; au-delà de **90 minutes** de trajet total, la note vaut 0. Une réservation provisoire (« hold ») dure jusqu'à l'heure d'arrivée prévue du bus + **5 minutes** de marge.

1.6 Hypothèses propres à l'optimisation (cas d'usage 5)

Seuil	Valeur	Usage
Facteur d'heure de pointe	125 % de la moyenne horaire	détecter une heure de pointe
Remplissage « saturé »	75 % (sens de pointe, sur 1h)	déclencher ajout de bus / horaires
Taux de refus élevé	10 % d'étudiants refusés	détecter la congestion
Attente longue	8 minutes en moyenne	détecter la congestion
Ligne sous-utilisée (fusion)	moins de 10 % de remplissage moyen	proposer une fusion de lignes
Heure creuse	remplissage moyen $\leq$ 30 %	basculer des bus vers la pointe
Remplissage cible	60 %	dimensionner les bus à garder en heures creuses
Fréquence minimale	1 bus / 30 min	ne jamais trop dégarnir une ligne en heures creuses
Tronçon trop long	plus de 3 km sans arrêt	proposer un nouvel arrêt
Part de marcheurs élevée	20 % marchent plus d'1 km	proposer un nouvel arrêt

2. Pour aller plus loin

Réponses aux cinq points demandés par l'énoncé : « Si vous aviez disposé de plus de temps, comment seriez-vous allé plus loin ? »

1. Recul sur vos choix — qu'auriez-vous fait différemment dès le départ ?

Quatre choix que je referais différemment, en sachant ce que je sais maintenant :

- **1. Modéliser les correspondances** — j'ai choisi un modèle « une ligne = un étudiant » pour tenir dans le temps imparti. C'est la simplification qui limite le plus la recommandation (cas d'usage 4) : un étudiant qui devrait combiner deux lignes n'obtient aucune proposition.
- **2. Séparer plus tôt simulateur et état de la flotte** — le moteur sert aujourd'hui à la fois de source de vérité et de générateur de données. Avec de vraies positions GPS, il faudrait un composant « état de la flotte » indépendant, que le simulateur ne ferait qu'alimenter en mode démonstration. Je l'aurais isolé derrière une interface dès le départ.
- **3. Écrire les invariants avant les fonctionnalités** — les tests de propriétés du moteur (capacité jamais dépassée, conservation des étudiants) m'ont fait gagner beaucoup de temps de débogage ; je les écrirais en tout premier la prochaine fois, avant même les services.
- **4. Utiliser un vrai routage** — un coefficient de détour de 1,3 est une approximation correcte mais grossière ; un service de routage (OSRM, GraphHopper) sur OpenStreetMap donnerait des distances et des temps de trajet bien plus fiables.

2. Dette technique — quelles dettes techniques avez-vous volontairement acceptées ?

Par ordre de gravité décroissante :

- **État en mémoire, un seul processus** — pas de persistance, pas de haute disponibilité. Acceptable pour un prototype de 72 heures, à corriger avant toute mise en production réelle.
- **Polling toutes les 2 secondes** — simple et fiable à petite échelle, mais coûteux en charge (voir point 3 ci-dessous).
- **Vitesse et durée d'arrêt fixes par heure** — pas d'apprentissage sur un historique réel de circulation.
- **Seuils et pondérations « à dire d'expert »** — les seuils de détection (config.js) et les pondérations de la recommandation ne sont pas calibrés sur des données de terrain.
- **Pas d'authentification ni de limitation de débit** — sur l'API, à ajouter avant toute exposition publique.
- **Pas de tests côté interface** — les tests couvrent le moteur, la recommandation et les analyses côté serveur, pas les composants React.
- **Carte schématique en SVG** — pas de fond de carte réel, pas de tracé routier exact.

3. Passage à l'échelle — 50 000 étudiants et 300 bus en heure de pointe : qu'est-ce qui casserait en premier ?

Mesuré : une recommandation coûte environ **0,07 ms** de calcul pour une flotte de 20 bus (npm run bench), et ce coût dépend des bus des lignes concernées, pas de la taille totale de la flotte. Avec 300 bus sur une quarantaine de lignes, on reste dans quelques dixièmes de milliseconde par requête : **ce n'est pas la recommandation qui casse en premier.**

Ce qui casserait, dans l'ordre :

- **1. Le polling des écrans temps réel** — si 10 000 étudiants gardent un écran ouvert avec un rafraîchissement toutes les 2 secondes, cela représente environ 5 000 requêtes par seconde, chacune sérialisant l'état de toute la flotte (~350 octets par bus mesurés, soit ~105 Ko pour 300 bus). C'est le premier goulot d'étranglement. Solution

: passer à un flux SSE ou WebSocket avec des données filtrées par client, et mettre en cache l'instantané de la flotte 1 à 2 secondes.

- **2. L'état en mémoire d'un seul processus** — impossible de répartir la charge sur plusieurs instances, et les réservations provisoires seraient perdues à chaque redémarrage. Solution : Redis pour les réservations (avec expiration automatique et décrémentation atomique du nombre de places, pour éviter que deux étudiants réservent la même dernière place).
- **3. Le calcul des analyses historiques** — recalculer les statistiques depuis zéro à chaque requête ne tient plus avec un vrai volume de données. Solution : PostgreSQL/PostGIS avec des agrégats horaires précalculés.
- **4. La qualité des prévisions** — avant même la performance, les temps d'arrivée basés sur une vitesse moyenne fixe deviendront visiblement faux avec de vrais bus ; il faudra des ETA appris sur l'historique, par tronçon et par heure.

4. Données manquantes — quelles données demanderiez-vous, et quelle décision permettraient-elles de prendre ?

Donnée à demander	Décision rendue possible
Comptages réels de montées/descentes par arrêt et par bus (1 semaine)	Valider ou corriger le profil horaire et le volume quotidien estimé (aujourd'hui 8 520/jour)
Positions GPS réelles et temps de parcours par tronçon et par heure	Remplacer les vitesses fixes par de vraies ETA, mesurer la ponctualité
Origine-destination des étudiants (résidence par quartier, emploi du temps)	Décider de nouvelles lignes ou de nouveaux arrêts (aujourd'hui seulement un point au milieu d'un tronçon)
Emploi du temps des cours (heures de début/fin par faculté)	Anticiper les pointes ; décaler des horaires de cours plutôt que d'ajouter des bus
Refus et abandons réels (étudiants renonçant, taxi-moto)	Mesurer la demande non servie, aujourd'hui invisible ; dimensionner la flotte sur le besoin réel
Contraintes d'exploitation (chauffeurs, pauses, coût au km)	Savoir si « basculer des bus vers la pointe » est faisable et combien cela coûte
Comportement face à la recommandation (taux d'acceptation, écart prévu/réel)	Calibrer les pondérations du score au lieu de les fixer à dire d'expert

5. Mise en production — quelle serait votre première itération avec de vrais étudiants et de vrais chauffeurs ?

Objectif : **prouver la valeur sur une seule ligne**, avec des données réelles, avant de généraliser.

- **1. Choisir une ligne pilote** — la plus saturée (L1 dans le modèle) : 3 bus, une pointe marquée, un seul campus — facile à observer et à mesurer.
- **2. Côté chauffeurs** — une application simple envoyant le GPS toutes les 5 secondes et permettant de saisir le nombre de passagers à chaque arrêt (comptage manuel d'abord, plus fiable qu'un capteur qui tombe en panne). Une journée de formation, un référent par bus.
- **3. Côté étudiants** — une page web mobile légère (sans installation, adaptée aux réseaux faibles) montrant les prochains bus à l'arrêt choisi, leur remplissage et la recommandation. La réservation de place n'est activée que lorsque la fiabilité des données GPS le permet.
- **4. Mesurer avant/après** — attente moyenne, étudiants laissés à quai, taux de remplissage en pointe et en creux, écart entre heure prévue et heure réelle : ces indicateurs répondent directement à l'objectif de l'énoncé (moins de bus sous-chargés, moins d'étudiants à quai).

- **5. Garde-fous** — mode dégradé si un bus est silencieux plus de 60 secondes (il n'est plus recommandé) ; message « estimation » plutôt qu'une fausse précision ; accès sans compte pour cette première itération ; journalisation des recommandations pour analyser les écarts.
- **6. Retour d'expérience** — réunion hebdomadaire avec chauffeurs et étudiants, recalibrage des pondérations après deux semaines de données réelles, extension aux lignes L2 et L3 seulement si les indicateurs s'améliorent.

Document généré à partir du dépôt du projet (README.md, section « Pour aller plus loin » et section « Hypothèses »). Voir le README pour le détail des formules, de l'API et des résultats de simulation.